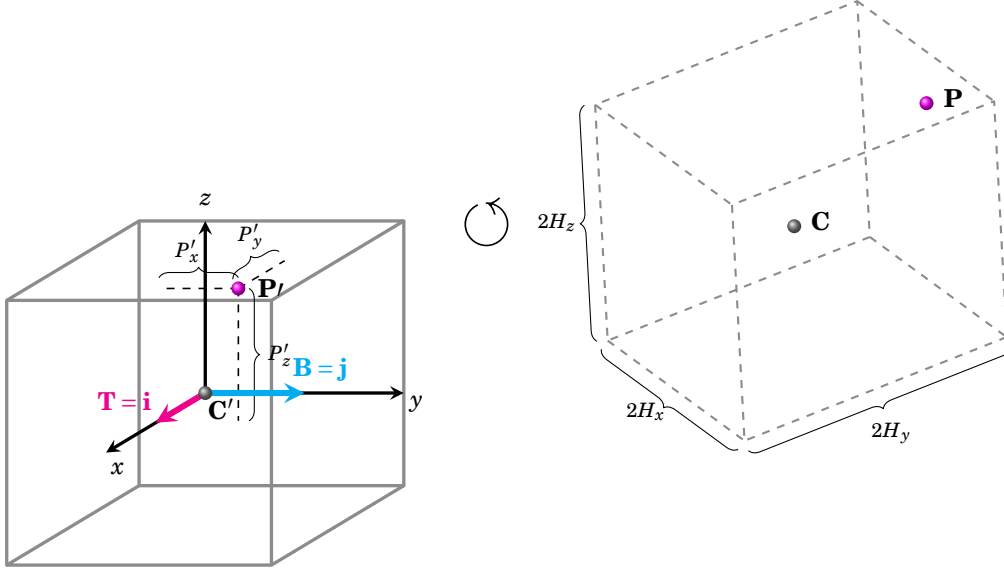


1. Problem Statement

Texture mapping involves the mapping of a 2D image onto the surface of a 3D shape – in this case a rectangular block. Before we do texture mapping, we “unorate” the cube (remove all its x, y, z rotations) and shift it to the origin, so that $C' = (0, 0, 0)$. This will make the mapping computations later easier. Then, we project the texture on each face separately.

Given a ray-cube intersection point P and its axis-aligned correspondence P' , the question we want to answer is that given the texture to project, what point (and by extension what color) of that texture corresponds to P' ? In other words, how does P map to texture coordinates?



We define:

- P the original point in world coordinates
- P' the origin-aligned unroated transform of P , i.e. $P' = R^T(P - C)$, so that $P_x \in [-H_x, H_x], P_y \in [-H_y, H_y], P_z \in [-H_z, H_z]$
- C' the center of the cube shifted to the origin, i.e. $C' = (0, 0, 0)$
- The half-width, half-height and half-depth of the block, H_x, H_y, H_z respectively

To help with mapping points from the 3D block to the texture, we define the following two vectors to orient the hit point:

- a *tangent direction vector* $\mathbf{T}$, to point along the horizontal direction and map to the texture's u direction.
- a *bitangent direction vector* $\mathbf{B}$, to point along the vertical direction and map to the texture's v direction.

2. Geometry

The normal N is often useful too and it's simply their cross product; $N = T \times B$. T and B vary depending on which surface is hit, since we're dealing with rectangular block, they are $\pm \mathbf{i}$ or $\pm \mathbf{j}$ depending on the face. They are direction vectors therefore their origin doesn't matter – they span the hit surface and show the directions of increasing u and v . The uv space, or *texture space*, is a 2D space from 0 to 1, where u is normalized by the texture image's width and v is normalized by its height.

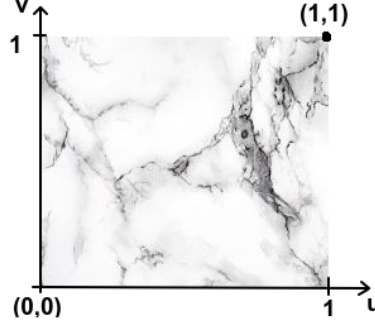


Fig. 1. The uv space, or texture space.

For any hit point $\mathbf{P}$ on that face, the texture coordinates are obtained by projecting $\mathbf{P}$ onto the $\mathbf{T}$ and $\mathbf{B}$ axes:

$$u = \frac{(\mathbf{P} - \mathbf{P}_0) \cdot \mathbf{T}}{\|\mathbf{T}\|^2}, \quad v = \frac{(\mathbf{P} - \mathbf{P}_0) \cdot \mathbf{B}}{\|\mathbf{B}\|^2}.$$

For our setup, this formula simplifies into the x and y components of $\pm \mathbf{P} - \mathbf{P}_0$ respectively. $\mathbf{P}_0$ is the *corner* of the hit face. Since we're dealing with rectangular faces, $\mathbf{P}_0$, $\mathbf{T}$, and $\mathbf{B}$ have a fixed definition for each face:

■ $+x$ face:

$$\mathbf{N} = (1, 0, 0), \quad \mathbf{T} = (0, 0, 1), \quad \mathbf{B} = (0, 1, 0), \quad \mathbf{P}_0 = (H_x, -H_y, -H_z).$$

■ $-x$ face:

$$\mathbf{N} = (-1, 0, 0), \quad \mathbf{T} = (0, 0, -1), \quad \mathbf{B} = (0, 1, 0), \quad \mathbf{P}_0 = (-H_x, -H_y, H_z).$$

■ $+y$ face:

$$\mathbf{N} = (0, 1, 0), \quad \mathbf{T} = (1, 0, 0), \quad \mathbf{B} = (0, 0, -1), \quad \mathbf{P}_0 = (-H_x, H_y, H_z).$$

■ $-y$ face:

$$\mathbf{N} = (0, -1, 0), \quad \mathbf{T} = (1, 0, 0), \quad \mathbf{B} = (0, 0, 1), \quad \mathbf{P}_0 = (-H_x, -H_y, -H_z).$$

■ $+z$ face:

$$\mathbf{N} = (0, 0, 1), \quad \mathbf{T} = (1, 0, 0), \quad \mathbf{B} = (0, 1, 0), \quad \mathbf{P}_0 = (-H_x, -H_y, H_z).$$

■ $-z$ face:

$$\mathbf{N} = (0, 0, -1), \quad \mathbf{T} = (-1, 0, 0), \quad \mathbf{B} = (0, 1, 0), \quad \mathbf{P}_0 = (H_x, -H_y, -H_z).$$

If texture *repetition* (tiling) is desired, we only keep the fractional part of u and v , i.e. we apply (more on that later):

$$u \leftarrow \text{frac}(u) = u - \lfloor u \rfloor, \quad v \leftarrow \text{frac}(v) = v - \lfloor v \rfloor.$$

3. Mapping from World Coordinates to Texture Coordinates

The next question is given the transformed $\mathbf{P}'$, how do we determine the face it belongs to? We need to know this to map the texture (u, v) to the correct face. This is simple since the surface hit will always have one coordinate maximized in terms of normalized absolute value (e.g. normalized $\pm H_x$, $\pm H_y$, or $\pm H_z$) and the other two will be smaller. Depending of the sign of the maximum absolute normalized coordinate, we can also determine which of the two possible faces is hit. For this, we define the normalized coordinates a_x, a_y, a_z and the distances to x, y, z axes $d_x = |a_x|, d_y = |a_y|, d_z = |a_z|$

$$a_x = \frac{P'_x}{H_x}, \quad a_y = \frac{P'_y}{H_y}, \quad a_z = \frac{P'_z}{H_z}, \quad d_x = |a_x|, \quad d_y = |a_y|, \quad d_z = |a_z|,$$

The largest of $\{d_x, d_y, d_z\}$ indicates which face is hit:

- $d_x = \max(d_x, d_y, d_z) \Rightarrow \pm x$ face,
- $d_y = \max(d_x, d_y, d_z) \Rightarrow \pm y$ face,
- $d_z = \max(d_x, d_y, d_z) \Rightarrow \pm z$ face.

The sign of a_x , a_y or a_z tells us whether we are on the positive or negative face.

The next step is to map d_x, d_y, d_z and the hit face to the uv space. Since $P'_i/H_i \in [-2, 2]$, to map to $u, v \in [0, 1]$, we take $\frac{P'_i}{2H_i} + \frac{1}{2}$. We can find out which combination of $\frac{P'_i}{2H_i}$ to take for u, v based on the face being hit. Notice how when we land in a face of opposite sign, we simple invert u and v , i.e. $u_- \leftarrow 1 - u_+$, or $v_- \leftarrow 1 - v_+$, depending on which face is hit.

- $+x$ face ($P'_x = +H_x$)

$$u = \frac{P'_z}{2H_z} + \frac{1}{2}, \quad v = \frac{P'_y}{2H_y} + \frac{1}{2}$$

- $-x$ face ($P'_x = -H_x$)

$$u = 1 - \frac{L_z}{2H_z} - \frac{1}{2}, \quad v = \frac{L_y}{2H_y} + \frac{1}{2} \quad (\text{u flip})$$

- $+y$ face ($P'_y = +H_y$)

$$u = \frac{L_x}{2H_x} + \frac{1}{2}, \quad v = \frac{L_z}{2H_z} + \frac{1}{2}$$

- $-y$ face ($P'_y = -H_y$)

$$u = \frac{L_x}{2H_x} + \frac{1}{2}, \quad v = 1 - \frac{L_z}{2H_z} - \frac{1}{2} \quad (\text{v flip})$$

- $+z$ face ($P'_z = +H_z$)

$$u = \frac{L_x}{2H_x} + \frac{1}{2}, \quad v = \frac{L_y}{2H_y} + \frac{1}{2}$$

- $-z$ face ($P'_z = -H_z$)

$$u = 1 - \frac{L_x}{2H_x} - \frac{1}{2}, \quad v = \frac{L_y}{2H_y} + \frac{1}{2} \quad (\text{u flip})$$

To repeat the texture by n times on each face, we simply multiply u, v by n before taking its fractional part. This leads to a periodic repeat with period n per face.

$$u_{\text{rep}} = \text{frac}(u \cdot n), \quad v_{\text{rep}} = \text{frac}(v \cdot n), \quad n = \text{repeat factor}$$

where $\text{frac}(x) = x - \lfloor x \rfloor$ is the fractional part.

Finally, convert $(u_{\text{rep}}, v_{\text{rep}})$ to integer coordinates (multiplying by the texture width and height) and sample the texture, i.e. $u' = u \cdot \text{texture width}$, $v' = v \cdot \text{texture height}$. Then we can return the color of the texture at coordinates u', v' .

We can now put all this together into some pseudocode.

4. Texture Block Mapping Pseudocode

Algorithm 1 TextueMapBlock

Require: Hit point $\mathbf{P}$, cube parameters $(\mathbf{C}, R, H_x, H_y, H_z)$, texture matrix T of size $W \times H$, repeat factor n

Ensure: Color value Color

```

1: if  $T$  is null then
2:   return cube base color
3:  $\mathbf{P}' \leftarrow R^\top \cdot \mathbf{P} - \mathbf{C}$  ▷ world  $\rightarrow$  local
4:  $(P'_x, P'_y, P'_z) \leftarrow \mathbf{P}'$ 
5:  $d_x \leftarrow |P'_x|/H_x$ 
6:  $d_y \leftarrow |P'_y|/H_y$ 
7:  $d_z \leftarrow |P'_z|/H_z$ 
8: if  $d_x \geq d_y$  and  $d_x \geq d_z$  then ▷  $\pm x$  face
9:    $u \leftarrow P'_z/(2H_z) + 0.5$ 
10:   $v \leftarrow P'_y/(2H_y) + 0.5$ 
11:  if  $P'_x < 0$  then
12:     $u \leftarrow 1 - u$ 
13: else if  $d_y \geq d_x$  and  $d_y \geq d_z$  then ▷  $\pm y$  face
14:   $u \leftarrow P'_x/(2H_x) + 0.5$ 
15:   $v \leftarrow P'_z/(2H_z) + 0.5$ 
16:  if  $P'_y < 0$  then
17:     $v \leftarrow 1 - v$ 
18: else ▷  $\pm z$  face
19:   $u \leftarrow P'_x/(2H_x) + 0.5$ 
20:   $v \leftarrow P'_y/(2H_y) + 0.5$ 
21:  if  $P'_z < 0$  then
22:     $u \leftarrow 1 - u$ 
23:  $u_{\text{scaled}} \leftarrow u \cdot n$  ▷  $n$  is the repeat factor
24:  $v_{\text{scaled}} \leftarrow v \cdot n$ 
25:  $u_{\text{rep}} \leftarrow u_{\text{scaled}} - \lfloor u_{\text{scaled}} \rfloor$ 
26:  $v_{\text{rep}} \leftarrow v_{\text{scaled}} - \lfloor v_{\text{scaled}} \rfloor$ 
27:  $t_x \leftarrow \lfloor u_{\text{rep}} \cdot W \rfloor$  ▷ texture image (integer) coordinate
28:  $t_y \leftarrow \lfloor v_{\text{rep}} \cdot H \rfloor$  ▷ texture image (integer) coordinate
29:  $t_x \leftarrow \min(\max(t_x, 0), W - 1)$ 
30:  $t_y \leftarrow \min(\max(t_y, 0), H - 1)$ 
31:  $\text{color} \leftarrow T[t_y, t_x]$ 
32: return color

```
